

NapCart Implementation Plan v1

Execution Plan for MVP Delivery

Version: v1.0 Date: 2026-05-28 Status: Draft Author: Codex with Naptime AI

1. Purpose

This document converts the approved NapCart planning and system-design artifacts into an execution-ready build plan.

Source artifacts:

- NapCart Project Roadmap
- NapCart PRD v1
- NapCart ERD v1
- NapCart Architecture Spec v1

Its job is to define:

- implementation phases
- milestone order
- ticket groups
- dependencies
- delivery sequence
- readiness criteria for starting development

2. Delivery Objective

The MVP goal is to deliver a production-ready first version of NapCart that allows:

- one restaurant to launch in Pakistan
- customers to place guest delivery and pickup orders
- branch-specific WhatsApp order routing
- branch staff to confirm or cancel through WhatsApp
- customer confirm/cancel notification
- management visibility through the admin dashboard

3. Delivery Principles

- Build in thin vertical slices
- Finish foundations before feature acceleration
- Keep the MVP aligned with approved scope only
- Prefer reusable modules over restaurant-specific hacks
- Treat mock WhatsApp mode as a first-class development tool
- Keep data, audit, and routing logic production-safe from the start

4. Delivery Model

Recommended delivery model:

- one codebase
- one MVP branch strategy
- one primary environment stack
- milestone-based execution
- task breakdown by module and dependency order

Recommended cadence:

- 1-week planning/setup sprint
- 4 to 6 core implementation sprints
- 1 stabilization and test sprint
- 1 deployment readiness sprint

Estimated MVP timeline:

- **7 to 10 weeks** depending on iteration speed and design churn

5. Implementation Overview

Phase 0 -> Project Setup and Delivery Foundations
Phase 1 -> Data Layer and Auth Foundations
Phase 2 -> Admin Core and Restaurant Configuration
Phase 3 -> Catalog, Branches, and Delivery Rules
Phase 4 -> Storefront, Cart, and Checkout
Phase 5 -> Order Engine and WhatsApp Mock Flow
Phase 6 -> Real WhatsApp Integration and Webhooks
Phase 7 -> Analytics, QA, Hardening, and Launch Readiness

6. Milestone Map

6.1 Milestone Sequence

M1 Foundations Ready
M2 Data Model and Admin Auth Ready
M3 Admin Management Core Ready
M4 Customer Ordering Flow Ready
M5 WhatsApp Confirmation Flow Ready
M6 MVP Stabilized
M7 Production Launch Ready

6.2 Milestone Visualization

M1 ----> M2 ----> M3 ----> M4 ----> M5 ----> M6 ----> M7
| | | | | |
setup db admin order whatsapp qa launch

7. Phase-by-Phase Plan

7.1 Phase 0: Project Setup and Delivery Foundations

Goal:

- prepare the repo, environments, and engineering structure before feature work

Primary outcome:

- the team can start building safely without re-deciding architecture every day

Tasks

- create the GitHub repository under `Naptime AI`
- initialize the Next.js + TypeScript project
- configure linting, formatting, and project scripts
- define the app folder/module structure
- create base environment variable strategy
- create Vercel project
- create Supabase project
- connect local development env
- set up Prisma

- define initial CI expectations
- create `.env.example`
- create shared constants for Pakistan defaults:
 - currency `PKR`
 - timezone `Asia/Karachi`
 - country/phone assumptions

Exit Criteria

- repo exists
- app boots locally
- Vercel project exists
- Supabase project exists
- Prisma can connect to database
- base project structure is approved

7.2 Phase 1: Data Layer and Auth Foundations

Goal:

- implement the approved schema and secure admin access foundation

Primary outcome:

- database and auth are ready for real feature development

Tasks

- translate ERD into Prisma schema
- create database migrations
- create seed strategy for local development
- implement restaurant, branch, settings, admin, catalog, customer, order, and WhatsApp tables
- set up Supabase Auth for admin accounts
- link auth identities to `admin_users`
- implement restaurant-scoped admin session resolution
- implement base repository/query layer
- create audit-friendly timestamps and transaction helpers

Exit Criteria

- migrations apply successfully
- seed data works
- admin can log in
- admin session resolves restaurant scope
- all core MVP tables exist and are queryable

7.3 Phase 2: Admin Core and Restaurant Configuration

Goal:

- give management the minimum control panel required to operate NapCart

Primary outcome:

- restaurant owner can configure operational settings and view the dashboard shell

Tasks

- build admin layout and navigation shell
- build dashboard home with summary cards placeholders
- implement restaurant branding settings
- implement branch CRUD
- implement operating hours UI
- implement open/closed and accepting-orders controls
- implement restaurant settings:
 - delivery enabled
 - pickup enabled
 - minimum order amount
 - global closed state
- implement WhatsApp connection settings UI
- implement product image and logo upload plumbing

Exit Criteria

- admin can manage restaurant identity
- admin can manage branches
- admin can manage operational availability

- admin can save WhatsApp configuration records

7.4 Phase 3: Catalog, Branches, and Delivery Rules

Goal:

- complete the core commerce configuration layer before storefront ordering starts

Primary outcome:

- a restaurant can define a usable menu and delivery rule set

Tasks

- implement category CRUD
- implement product CRUD
- implement variant CRUD
- implement add-on group CRUD
- implement add-on CRUD
- implement product availability toggles
- implement delivery/pickup availability toggles
- implement branch assignment display behavior
- implement delivery zone CRUD
- implement delivery fee configuration
- implement minimum-order validation rules
- implement storefront-ready menu query services

Exit Criteria

- menu is fully manageable from admin
- delivery zones and fees are configurable
- storefront can retrieve structured menu data from backend

7.5 Phase 4: Storefront, Cart, and Checkout

Goal:

- build the complete customer-facing ordering experience

Primary outcome:

- a customer can browse, select, and place a valid order

Tasks

- build storefront landing shell
- build branch selection flow
- build menu listing and product detail interactions
- build cart state management
- build variation and add-on selection
- build fulfillment selection:
 - delivery
 - pickup
- build guest checkout form
- implement Pakistan-friendly phone handling UX
- implement free-text address capture
- implement delivery notes field
- implement checkout validation UX
- build order placed / awaiting confirmation page

Exit Criteria

- customer can complete guest checkout
- all required order data reaches the backend correctly
- invalid carts are blocked before placement

7.6 Phase 5: Order Engine and WhatsApp Mock Flow

Goal:

- make ordering operationally real before using live provider integration

Primary outcome:

- orders are created reliably and can travel through the full confirm/cancel logic in mock mode

Tasks

- implement order creation service
- implement customer find-or-create logic
- implement transactional order persistence

- implement item/add-on snapshot logic
- implement `pending_confirmation` initialization
- implement order number generation
- implement status history writes
- implement WhatsApp adapter interface
- implement mock WhatsApp adapter
- implement outbound message log creation
- implement simulated confirm/cancel action flow
- implement customer notification service contract

Exit Criteria

- order creation works end to end
- mock branch confirmation updates order state
- mock cancellation updates order state
- logs and audit history are written correctly

7.7 Phase 6: Real WhatsApp Integration and Webhooks

Goal:

- replace simulation with real provider integration while preserving the same business contract

Primary outcome:

- branch staff can confirm or cancel from real WhatsApp

Tasks

- implement Meta WhatsApp Cloud API adapter
- implement outbound interactive order message format
- implement webhook verification endpoint
- implement inbound webhook parser
- implement provider-to-order correlation logic
- implement customer confirmation message send
- implement customer cancellation message send
- implement provider error handling
- implement WhatsApp settings validation
- verify branch-specific routing behavior

Exit Criteria

- order message reaches correct branch number
- branch action updates backend status
- customer receives matching response
- inbound and outbound logs are stored

7.8 Phase 7: Analytics, QA, Hardening, and Launch Readiness

Goal:

- stabilize the MVP and make it safe to deploy for a first restaurant

Primary outcome:

- MVP is testable, reviewable, and launch-ready

Tasks

- implement admin orders list
- implement admin order detail page
- implement customer list
- implement basic analytics summary:
 - total orders
 - confirmed
 - cancelled
 - pending confirmation
 - total sales
- implement critical empty states and error states
- add validation hardening
- add audit/log review helpers
- test branch availability and delivery rule edge cases
- test image uploads
- test auth boundaries
- test Pakistan phone normalization assumptions
- prepare staging review checklist
- prepare production env checklist

Exit Criteria

- MVP passes core functional QA
- launch checklist is complete
- first restaurant onboarding path is documented

8. Ticket Structure

Recommended ticket hierarchy:

```
Epic
  -> Feature Group
    -> Implementation Task
      -> QA / Verification Task
```

Example Epic Breakdown

```
Epic 1: Platform Foundations
Epic 2: Admin Configuration
Epic 3: Catalog and Delivery Rules
Epic 4: Storefront Ordering Flow
Epic 5: Order Engine
Epic 6: WhatsApp Operations
Epic 7: QA and Launch Readiness
```

9. Detailed Ticket Groups

9.1 Epic 1: Platform Foundations

- initialize codebase
- connect Vercel and Supabase
- configure Prisma
- add shared app config
- add auth foundation
- add file storage foundation

9.2 Epic 2: Admin Configuration

- admin shell

- restaurant settings
- branch management
- operating hours
- open/closed controls
- WhatsApp settings

9.3 Epic 3: Catalog and Delivery Rules

- categories
- products
- variants
- add-on groups
- add-ons
- product media
- delivery zones
- minimum order rules

9.4 Epic 4: Storefront Ordering Flow

- branch selection
- menu browsing
- cart
- checkout
- phone and address handling
- order placed state

9.5 Epic 5: Order Engine

- order transaction
- customer matching
- snapshotting
- order status history
- audit writes

9.6 Epic 6: WhatsApp Operations

- mock adapter
- provider adapter
- outbound order messages
- inbound webhook actions
- customer confirm/cancel notifications
- message logs

9.7 Epic 7: QA and Launch Readiness

- analytics summary
- admin review flows
- staging QA
- production readiness
- onboarding checklist

10. Dependency Logic

10.1 Critical Dependency Chain

```
Repo Setup
  -> Database + Auth
    -> Admin Settings + Branches
      -> Catalog + Delivery Rules
        -> Storefront + Checkout
          -> Order Engine
            -> WhatsApp Mock
              -> Real WhatsApp
                -> QA + Launch
```

10.2 Key Rules

- do not start real WhatsApp integration before the mock flow works
- do not start checkout finalization before delivery rules exist
- do not start storefront integration before menu queries are stable
- do not start production deployment before role/scope security is verified

11. Testing Strategy by Phase

11.1 Unit/Logic Tests

Focus on:

- price calculations
- delivery fee logic
- minimum order checks
- phone normalization helpers
- order status transitions
- WhatsApp action parsing

11.2 Integration Tests

Focus on:

- order creation transaction
- customer matching
- admin auth scoping
- branch-specific WhatsApp routing
- webhook confirm/cancel handling

11.3 Manual QA

Focus on:

- full storefront flow
- branch selection correctness
- delivery vs pickup behavior
- product availability edge cases
- admin operations
- failed WhatsApp send scenarios

12. Definition of Done

A task is done only when:

- implementation is complete
- validation rules are included
- restaurant scoping is respected
- happy path is verified

- main failure path is handled
- relevant tests or manual verification are performed
- no approved-scope requirement was silently changed

13. Launch Readiness Checklist

- repository and environments are stable
- database migrations are clean
- admin auth works in production
- product images and logo uploads work
- branch routing is verified
- delivery and pickup both work
- order logs are visible
- WhatsApp confirm/cancel flow is verified
- customer confirm/cancel message is verified
- management dashboard shows correct metrics
- first restaurant configuration can be completed without code edits

14. Risks During Implementation

14.1 Scope Creep

Risk:

- requests for payments, coupons, or rider flows may interrupt MVP

Mitigation:

- keep P0/P1/P2 boundaries visible in every sprint

14.2 WhatsApp Provider Delays

Risk:

- real provider setup may lag behind development

Mitigation:

- treat mock mode as a required implementation milestone, not a fallback afterthought

14.3 Weak Tenant Isolation

Risk:

- admin functionality may accidentally leak restaurant data

Mitigation:

- validate scope at repository and service layers early

14.4 Data Quality Issues

Risk:

- Pakistani phone formatting and free-text addresses may produce inconsistent records

Mitigation:

- normalize phone numbers consistently
- keep address parsing simple in MVP
- preserve raw input where needed

15. Recommended Immediate Next Execution Step

After this implementation plan is approved, the next action should be:

Set up the actual project foundation

That means:

- create GitHub repo
- create Supabase project
- create Vercel project
- initialize the Next.js app
- implement Prisma schema foundation

16. Final Recommendation

NapCart should now stop expanding planning artifacts and begin controlled execution.

The approved sequence from here is:

Implementation Plan
-> Project Setup

- > Schema + Auth
- > Admin Config
- > Storefront
- > Order Engine
- > WhatsApp
- > QA
- > Launch

This is the cleanest path to move from approved design into a real working MVP without losing focus.